

Calcolo Parallelo e Distribuito

Esempio uso del costrutto work-sharing: sections

prova esame tipo

Docente: Prof. L. Marcellino

Tutor: Prof. P. De Luca



INFORMAZIONI IMPORTANTI PER L'ESAME:

- **PROVA scritta: durata 2 ore**

Descrizione di un algoritmo parallelo relativo ad un problema assegnato, Calcolo Teorico di una o più metriche di valutazione (Speedup ed Efficienza, Isoefficienza, Ware-Amdhal), **decrizione e commento del codice relativo all'algoritmo**

- **PROVA orale (obbligatoria), ma solo se ammessi (scritto sufficiente)**

Traccia **tipo** del compito più o meno completa

Dati $A \in \mathbb{R}^{N\times M} b \in \mathbb{R}^{M}, a \in \mathbb{R}^{N}$ consideriamo il problema:

$$r = \alpha A \cdot b + \beta a,$$

con $\alpha, \beta \in \mathbb{N}$

- Assumendo l'utilizzo di **np** thread, determinare Speedup e Overhead, secondo la definizione classica, dell'intera procedura parallela, motivando tutti i passaggi. **(15pt)**
- Considerato il ***seguente*** codice che risolve il problema illustrato, 2-3 domande tecniche su openMP **(totale 5pt)**
- Fissati **N=12, M=26,** calcolare lo Speed up con la legge di Ware-Amdhal del nucleo computazionale: $A \cdot b$, quando **np=4=2x2** (**10pt)**

OSSERVAZIONI

Traccia **tipo** del compito più o meno completa

Dati $A \in \mathbb{R}^{N \times M} b \in \mathbb{R}^{M}, a \in \mathbb{R}^{N}$ consideriamo il problema:

$$r = \alpha A \cdot b + \beta a,$$

con $\alpha, \beta \in \mathbb{N}$



Le due operazioni sono completamente indipendenti!

Direttive

La **direttiva**

Si usano per creare un team e stabilire quali istruzioni devono essere eseguite in parallelo e come queste devono essere distribuite tra i thread del team creato

#pragma omp ***costrutto*** *[clause], [clause] … new-line*

oltre al costrutto **parallel** prevede anche:

- **tre** tipi di costrutti detti ***WorkSharing*** perché si occupano della distribuzione del lavoro al team di thread: **for**, **sections**, **single**
- Anche all'uscita da un costrutto work-sharing è sottintesa una barriera di sincronizzazione, se non diversamente specificato dal programmatore.

Direttive

- Il costrutto ***for*** specifica che le iterazioni del ciclo contenuto debbano essere distribuite tra i thread del team (secondo un ordine che non specificherò in dettaglio, come ho fatto con la somma; ma userò le potenzialità di OpenMP!)

#pragma omp for *[clause], [clause] ... new-line*
{

}
clause: **private**(list)
firstprivate(list)
lastprivate(list)
reduction(operator: list)
schedule(kind[, chunk_size])
collapse(n)
ordered
nowait

schedule
solo per questo
costrutto

Direttive

- Il costrutto ***single*** specifica che il blocco di istruzioni successivo verrà eseguito da un solo thread QUALSIASI del team

#pragma omp single *[clause], [clause] ... new-line*
{

}
clause: **private**(list)
firstprivate(list)
copyprivate(list)
nowait

Gli altri thread attendono che questo termini la sua porzione di codice

Direttive

costrutti ***WorkSharing: SECTIONS***

- Il costrutto ***sections*** conterrà un insieme di costrutti *section* ognuno dei quali verrà eseguito da un thread del team



Le diverse sezioni devono poter essere eseguite in ordine arbitrario

```
#pragma omp sections [clause], [clause] ... new-line
{
[#pragma omp section new-line]

[#pragma omp section new-line]
...
}
clause: private(list)
firstprivate(list)
lastprivate(list)
reduction(operator: list)
nowait
```



Rischio di sbilanciamento del carico

Tutti i costrutti **WorkSharing** possono essere combinati con il costrutto **parallel**, e le clausole ammesse sono l'unione di quelle ammesse per ognuno.

Il parallelismo delle architetture MIMD-SM

metodi numerici paralleli

Modello Numerico
$M_h(P)$

Ripartire dal modello numerico (modello matematico discretizzato **h=1,…,N**) e analizzare gli **N** passi in modo da distribuirli, eventualmente, a più unità processanti.

Più possibilità:

- **ogni unità** processante esegue un passo **differente**
  (**decomposizione funzionale**)
- **tutte le unità** processanti eseguono **la stessa** operazione su un sottoinsieme di dati
  (**decomposizione del dominio**)

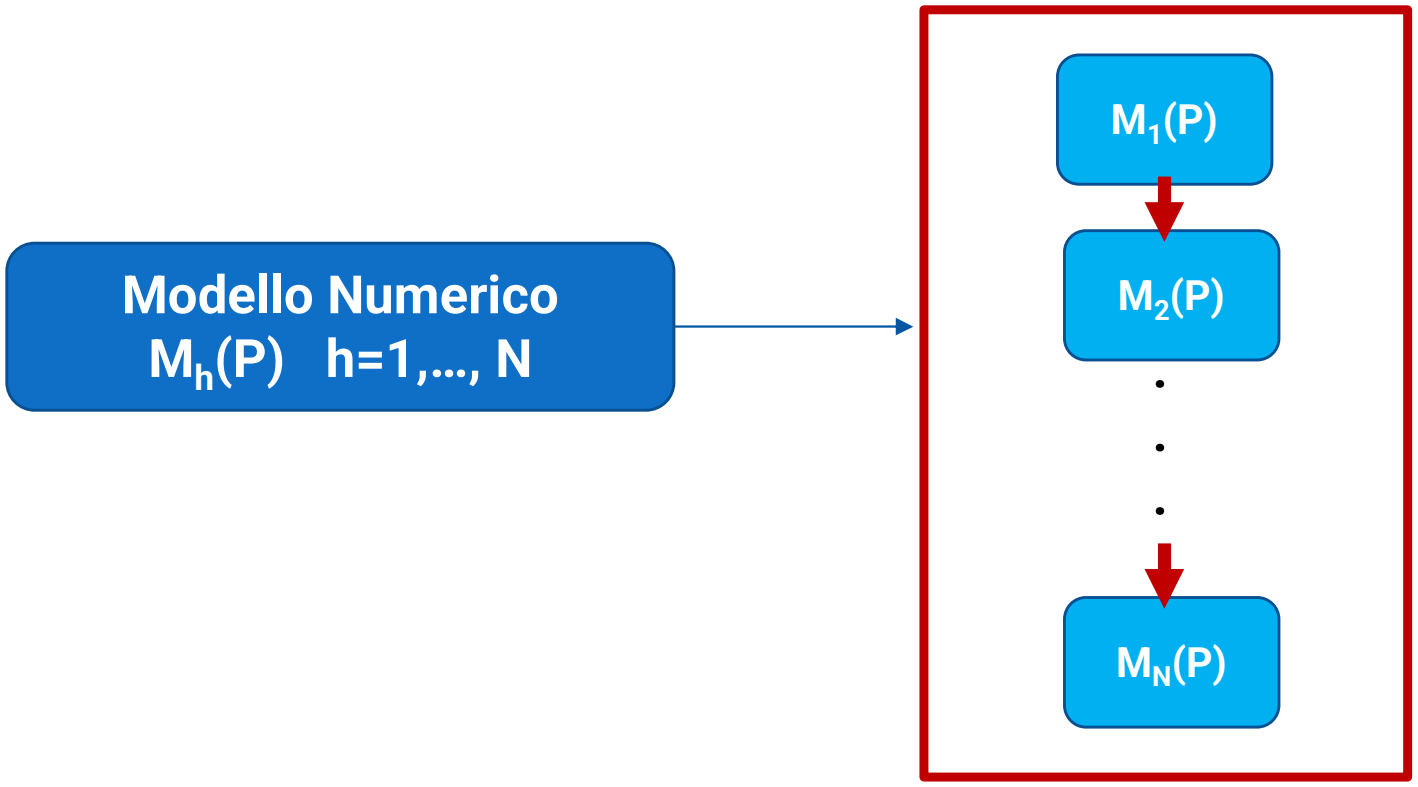
- **combinazione delle due possibilità precedenti**

Il parallelismo delle architetture MIMD-SM

metodi numerici paralleli

Modello Numerico
$M_h(P)$

Ripartire dal modello numerico (modello matematico discretizzato **h=1,…,N**) e analizzarlo per individuare **task indipendenti che possano essere processati separatamente e contemporaneamente.**

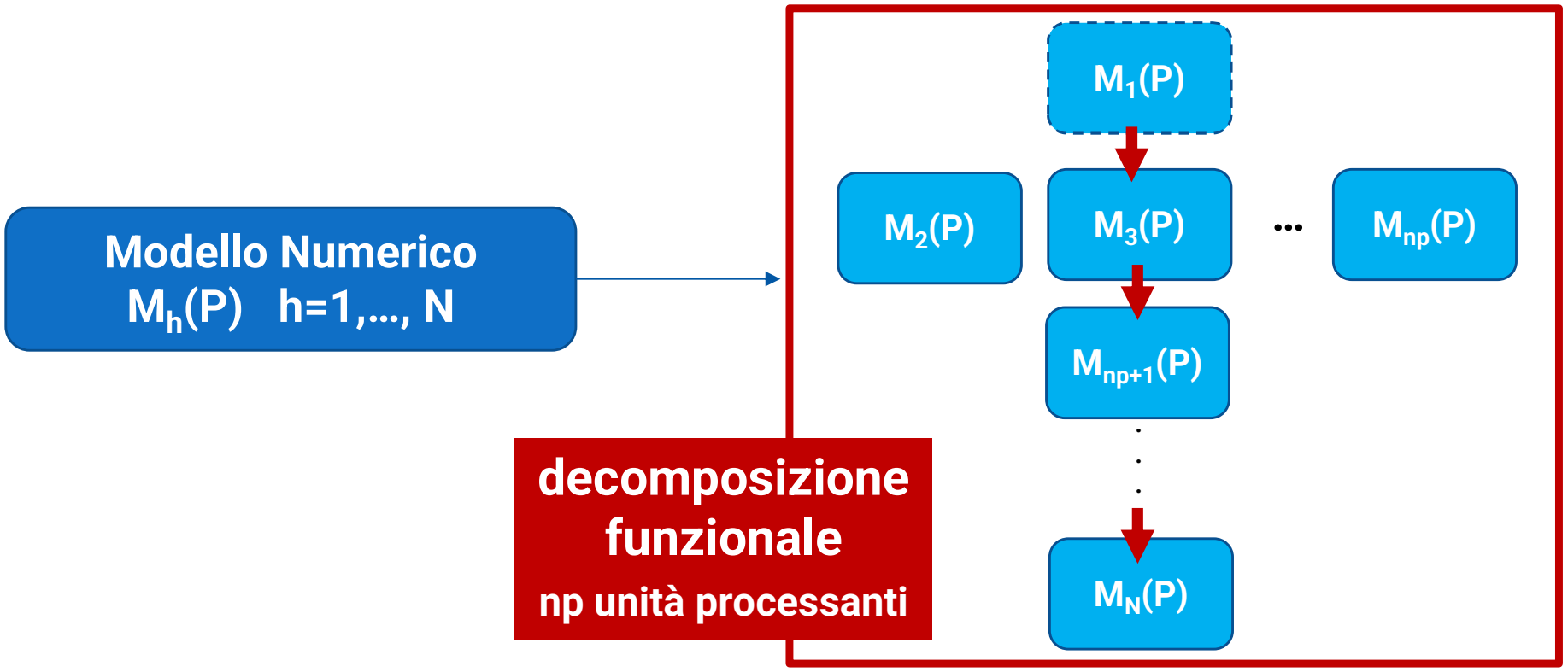


Il parallelismo delle architetture MIMD-SM

metodi numerici paralleli

Modello Numerico
$M_h(P)$

Ripartire dal modello numerico (modello matematico discretizzato **h=1,…,N**) e analizzarlo per individuare **task indipendenti che possano essere processati separatamente e contemporaneamente**.



Il parallelismo delle architetture MIMD-SM

metodi numerici paralleli – decomposizione funzionale

Ripartire dal modello numerico (modello matematico discretizzato **h=1,…,N**) e analizzarlo per individuare **task indipendenti che possano essere processati separatamente e contemporaneamente**.

Decomposizione funzionale = **eseguire elaborazioni differenti e indipendenti contemporaneamente**

Il modello è decomposto in base al lavoro che deve essere svolto.

Caratteristica fondamentale:

applicabile a un modello caratterizzato da più nuclei computazionali

Direttive

- Il costrutto ***sections*** conterrà un insieme di costrutti *section* ognuno dei quali verrà eseguito da un thread del team



Le diverse sezioni devono poter essere eseguite in ordine arbitrario

#pragma omp parallel sections *[clause], [clause] … new-line*
{
[#pragma omp section new-line]

[#pragma omp section new-line]
…
}
clause: **private**(list)
firstprivate(list)
lastprivate(list)
reduction(operator: list)
nowait



Rischio di sbilanciamento del carico

Come assegnare lavori differenti a differenti thread

```
#pragma omp parallel sections num_threads(3)
{
        #pragma omp section
         {
                printf( Hello world ONE! )
        }
        #pragma omp section
         {
                printf( Hello world TWO! )
        }

        #pragma omp section
         {
                printf( Hello world THREE! )
        }
}
```

Tre thread che fanno contemporaneamente tre cose diverse

Attenzione:

Tre thread che fanno contemporaneamente la stessa cosa.

```
#pragma omp parallel num_threads(3)
{
        printf( Hello world ! )
}
```

```
#pragma omp parallel sections num_threads(3)
{
        #pragma omp section
        {
                printf( Hello world ONE! )
        }
        #pragma omp section
        {
                printf( Hello world TWO! )
        }

        #pragma omp section
        {
                printf( Hello world THREE! )
        }
}
```

Tre thread che fanno contemporaneamente tre cose diverse

Che succede se cambio il numero di thread in questo codice?

```
#pragma omp parallel sections num_threads(4)
{
        #pragma omp section
        {
                printf( Hello world ONE! )
        }
        #pragma omp section
        {
                printf( Hello world TWO! )
        }

        #pragma omp section
        {
                printf( Hello world THREE! )
        }
}
```

Un quarto thread è allocato, ma non fa niente!!!

E se ne metto meno?

```
#pragma omp parallel sections num_threads(2)
{
        #pragma omp section
        {
                printf( Hello world ONE! )
        }
        #pragma omp section
        {
                printf( Hello world TWO! )
        }

        #pragma omp section
        {
                printf( Hello world THREE! )
        }
}
```

Un thread fa il doppio lavoro

Traccia tipo del compito più o meno completa

Dati $A \in \mathbb{R}^{N\times M} b \in \mathbb{R}^{M}, a \in \mathbb{R}^{N}$ consideriamo il problema:

$$r = \alpha A \cdot b + \beta a,$$

con $\alpha, \beta \in \mathbb{N}$

- Assumendo l'utilizzo di **np** thread, determinare Speedup e Overhead, secondo la definizione classica, dell'intera procedura parallela, motivando tutti i passaggi. **(15pt)**
- Considerato il ***seguente*** codice che risolve il problema illustrato, 2-3 domande tecniche su openMP **(totale 5pt)**
- Fissati **N=12, M=26,** calcolare lo Speed up con la legge di Ware-Amdhal del nucleo computazionale: $A \cdot b$, quando **np=4=2x2** **(10pt)**

$$\boldsymbol{T_1^{TOT}(N,M) = N[2 \cdot M - 1] + N + N + N}$$

IN PARALLELO HO ALMENO 2 POSSIBILITA':
osservazioni sulla previsione di scelta!

$$T_1^{TOT}(N, M) = N[2 \cdot M - 1] + 3N$$

Dati $A \in \mathbb{R}^{N \times M}, b \in \mathbb{R}^{M}, a \in \mathbb{R}^{N}$ consideriamo il problema:

$$r = \alpha A \cdot b + \beta a,$$

con $\alpha, \beta \in \mathbb{N}$

Decomposizione funzionale:

- **1 thread**, determina: $A \cdot b$ - **1 thread** calcola: βa,

  CONTEMPORANEAMENTE (conto solo chi fa più cose)

  $$T_2^{TOT}(N, M) = N[2 \cdot M - 1] + \cdots$$

- **2 thread** si dividono $[\alpha A \cdot b]$ più la somma finale tra vettori **(freccia nera)**…

  $$\ldots + \frac{N}{2} + \frac{N}{2}$$

$$T_2^{TOT}(N, M) = N[2 \cdot M - 1] + 2\frac{N}{2}$$

$$T_1^{TOT}(N, M) = N[2 \cdot M - 1] + 3N$$

Dati $A \in \mathbb{R}^{N \times M}, b \in \mathbb{R}^{M}, a \in \mathbb{R}^{N}$ consideriamo il problema:

$$r = \alpha A \cdot b + \beta a,$$

con $\alpha, \beta \in \mathbb{N}$

Decomposizione del dominio:

- **np thread**, determinano: $\alpha A \cdot b$ **(scelgo la 1 strategia)**

$$T_{np}^{TOT}(N, M) = \frac{N}{np}[2 \cdot M - 1] + \frac{N}{np} + \cdots$$

- **np thread**, calcolano: βa **(banalmente la full parallel)**

$$\ldots + \frac{N}{np}$$

- **np thread**, fanno la somma finale **(un'altra full parallel)**

$$\ldots + \frac{N}{np}$$

$$T_{np}^{TOT}(N, M) = \frac{N}{np}[2 \cdot M - 1] + 3\frac{N}{np}$$

Traccia tipo del compito più o meno completa

Dati $A \in \mathbb{R}^{N\times M} b \in \mathbb{R}^{M}, a \in \mathbb{R}^{N}$ consideriamo il problema:

$$r = \alpha A \cdot b + \beta a,$$

con $\alpha, \beta \in \mathbb{N}$

- Assumendo l'utilizzo di **np** thread, determinare Speedup e Overhead, secondo la definizione classica, dell'intera procedura parallela, motivando tutti i passaggi. **(15pt)**
- Considerato il ***seguente*** codice che risolve il problema illustrato, 2-3 domande tecniche su openMP **(totale 5pt)**
- Fissati **N=12, M=26,** calcolare lo Speed up con la legge di Ware-Amdhal del nucleo computazionale: $A \cdot b$, quando **np=4=2x2** **(10pt)**

Calcoli in aula!

Traccia tipo del compito più o meno completa

Dati $A \in \mathbb{R}^{N\times M} b \in \mathbb{R}^{M}, a \in \mathbb{R}^{N}$ consideriamo il problema:

$$r = \alpha A \cdot b + \beta a,$$

con $\alpha, \beta \in \mathbb{N}$

- Assumendo l'utilizzo di **np** thread, determinare Speedup e Overhead, secondo la definizione classica, dell'intera procedura parallela, motivando tutti i passaggi. **(15pt)**
- Considerato il ***seguente*** codice che risolve il problema illustrato, 2-3 domande tecniche su openMP **(totale 5pt)**
- Fissati **N=12, M=26,** calcolare lo Speed up con la legge di Ware-Amdhal del nucleo computazionale: $A \cdot b$, quando **np=4=2x2** **(10pt)**

Infinite possibilità....

```
#pragma omp parallel for private(j)
for (i = 0; i < N; i++) {
    tmp[i] = 0.0;
    for (j = 0; j < M; j++)
        tmp[i] += A[i * M + j] * b[j];

    tmp[i] = alpha * tmp[i];
}

#pragma omp parallel for
for (i = 0; i < N; i++)
    y[i] = beta * a[i];

#pragma omp parallel for
for (i = 0; i < N; i++)
    y[i] = tmp[i] + y[i];
```

- Quali devono essere le variabili shared e quali private per le tre direttive? **(2pt)**
- Illustrare il funzionamento dei costrutti impiegati (**3pt)**

TUTTO QUELLO CHE C'È NELLE SLIDE!